Module 2

IEEE 1364-2001 (Verilog-2001)

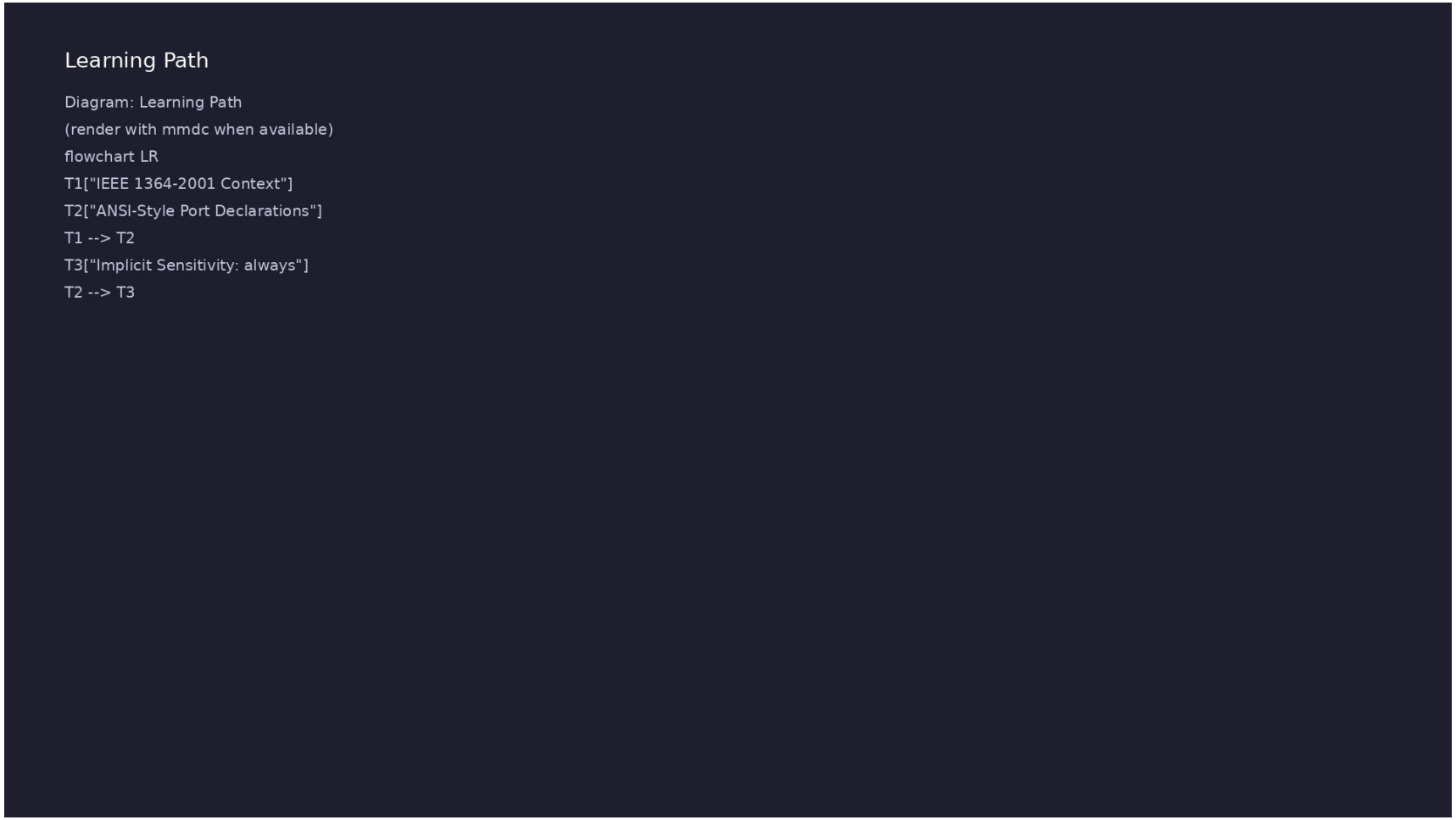
Learning objectives

- Master the major Verilog update (IEEE 1364-2001) and its additions over 1364-1995: ANSI ports, impl...

Prerequisites

- See module README

Learning path



Master the major Verilog update (IEEE 1364-2001) and its additions over 1364-1995: ANSI ports, implicit sensitivity, generate, signed...

Overview

- This module covers IEEE Std 1364-2001 (Verilog-2001), the first major revision of the Verilog stand…

How to learn this module

Suggested learning path (1/2)

- Skim this document — Goal, Overview, and Topics Covered set the IEEE scope for Module 2.
- Study design architecture — See how DUTs, examples, and testbenches fit together under `module2/`.
- Work through labs in order — Open `module2/EXAMPLES.md` and run each `make clean && make run` from…
- Run the full module script — `./scripts/module2.sh` from the repo root to simulate all examples and…
- Complete the exercises — Try each exercise in this document before reading solutions or peeking at…

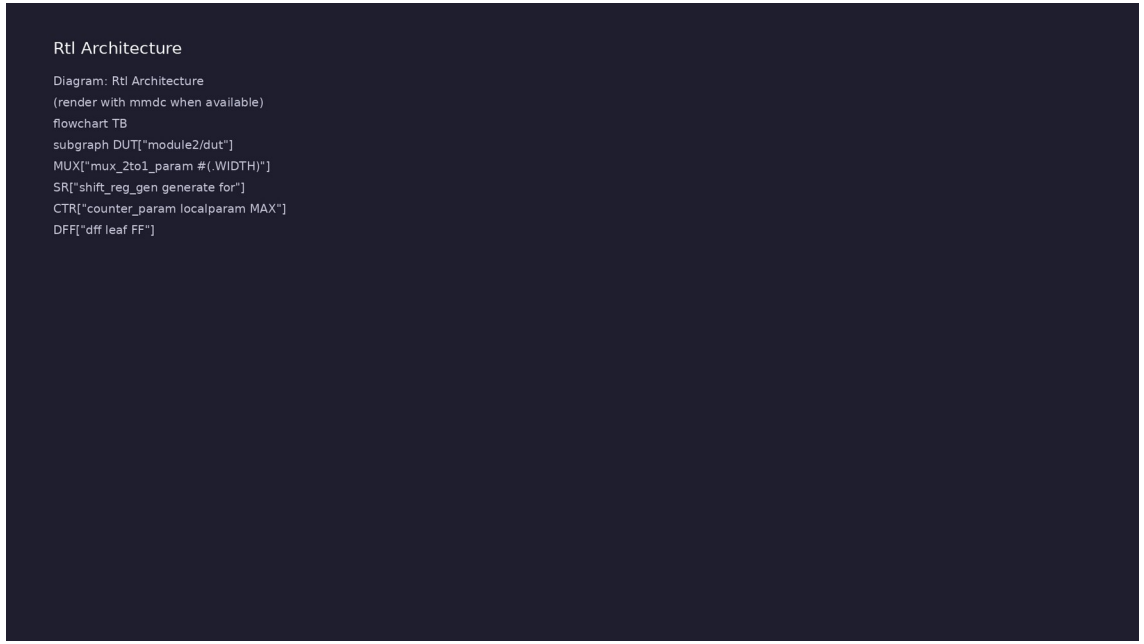
Suggested learning path (2/2)

- Review Common Pitfalls — Can you explain each mistake and the fix?
- Self-check — Use `module2/CHECKLIST.md` before starting the next module.

Design architecture

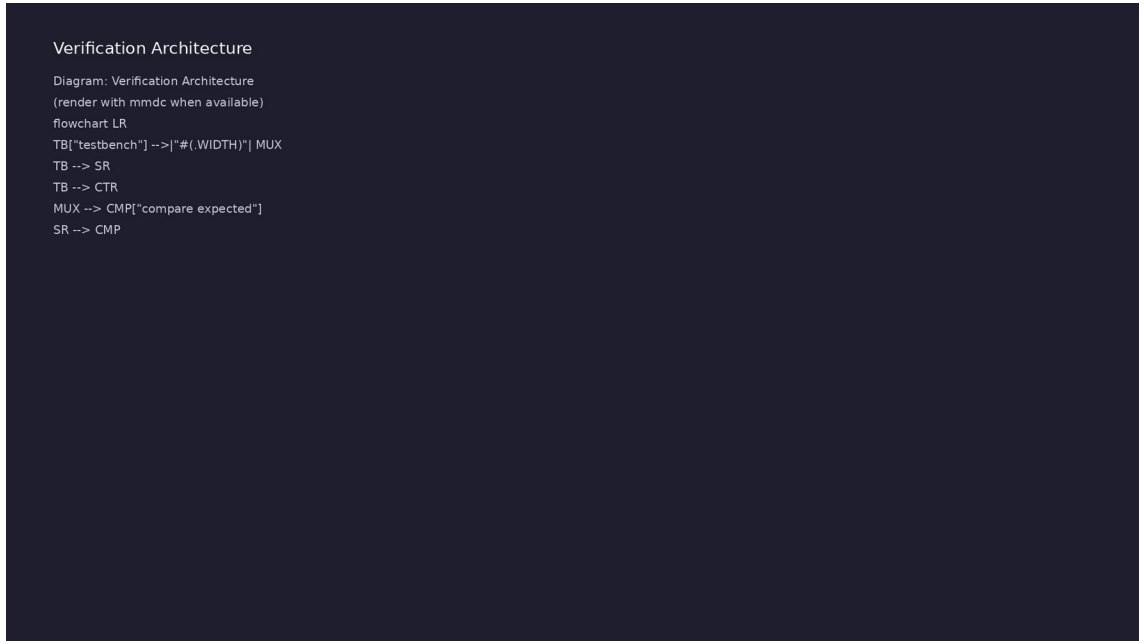
1. Repository layout (module2/)

- Builds on Module 1 — same examples + tests pattern under module2/
- Examples highlight 2001 additions: ANSI ports, always @*, generate, signed
- module2/dut/ — reference gates and parameterized blocks
- scripts/module2.sh — batch run all 2001-feature labs



2. 2001 RTL patterns vs 1995

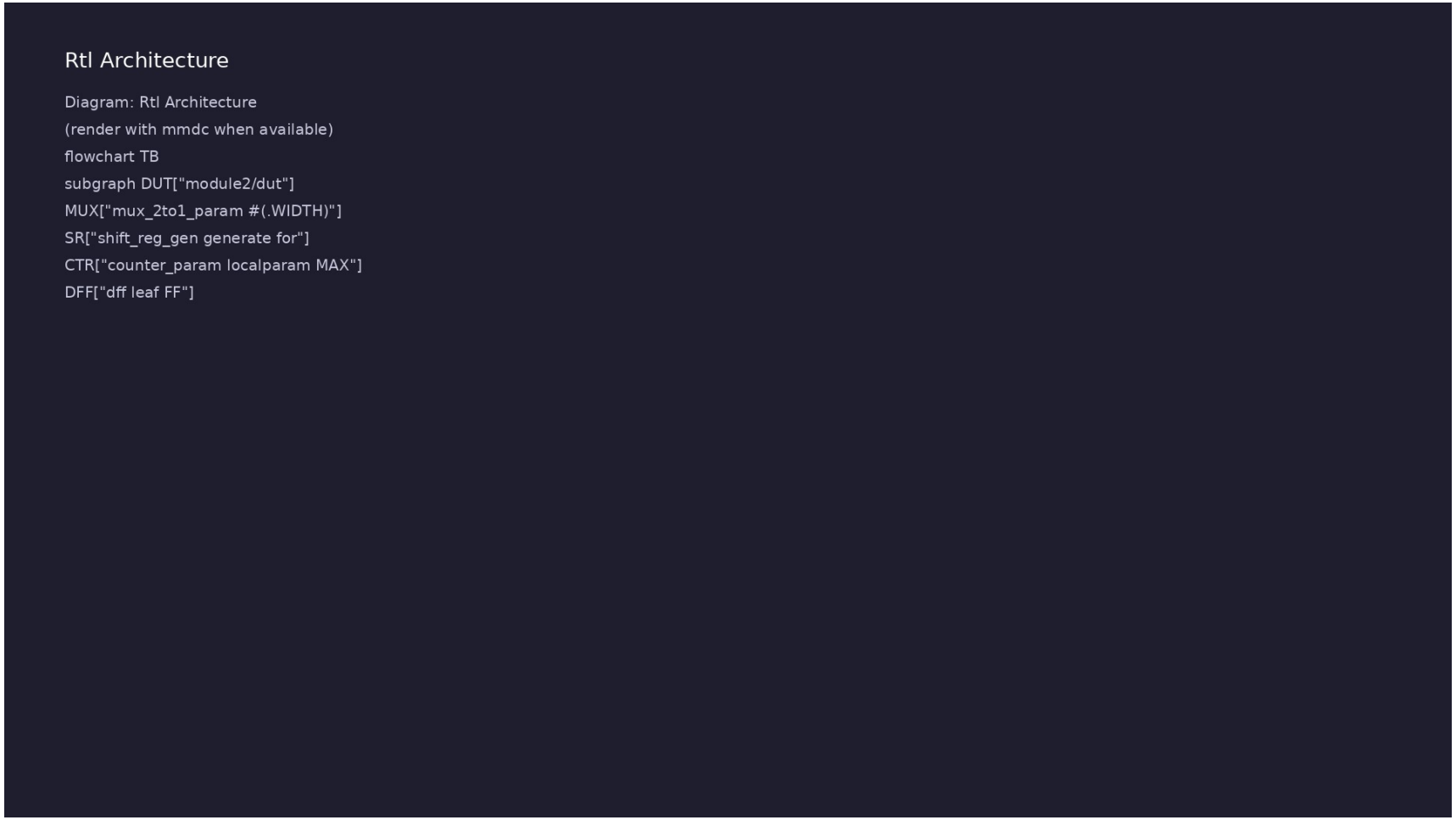
- ANSI ports combine direction, type, and width in the port list
- always @* replaces manual sensitivity lists for combinational logic
- generate if/for creates replicated or conditional hardware
- signed keyword and $signed enable two's-complement arithmetic



3. Hierarchy with generate

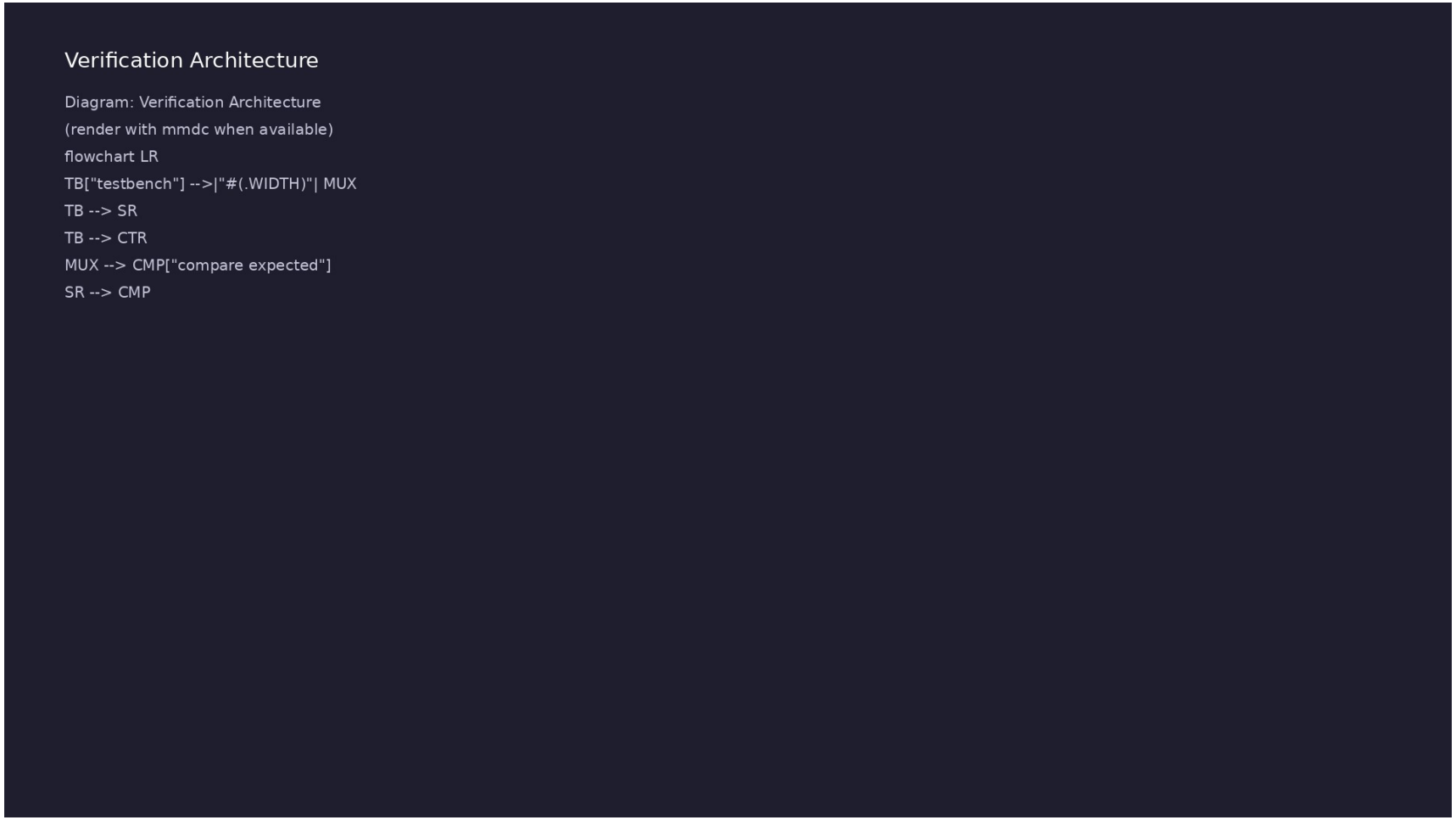
- param_mux and ripple_adder show parameterized instantiation
- gen_if demonstrates conditional generate blocks
- Parent passes parameters; child modules scale without copy-paste

RTL block diagram (reference)



Module 2: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)



Module 2: stimulus, observation, and checking.

ANSI ports — mux2 (2001)

```
/*
 * 2:1 multiplexer - IEEE 1364-2001 ANSI style
 *
 * Port direction and type in port list (no separate declarations).
 */

module mux2_2001 (
    input  wire a,
    input  wire b,
    input  wire sel,
    output reg  y
);
    always @(a or b or sel)
        y = sel ? b : a;
endmodule
```

```
# ...
```

always @* combinational block

```
/*
 * always @* - IEEE 1364-2001 implicit sensitivity
 *
 * Sensitive to every signal read in the block; no manual list.
 */

module comb_demo (
    input  wire a,
    input  wire b,
    input  wire c,
    input  wire sel,
    output reg  y_comb,
    output reg  mux_out
);
    /* Implicit sensitivity: tool infers a, b, c and sel, a, b */
    always @* begin
# ...
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module2 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Key files to study

Open these in the repo

- module2/examples/ansi_ports/mux2.v — ANSI port list pattern
- module2/examples/procedural/always_star.v — implicit sensitivity
- module2/examples/generate/param_mux.v — generate for loop
- module2/examples/signed/adder_signed.v — signed arithmetic
- scripts/module2.sh — full module execution

Verification & testing methods

1. Running labs and tests

- Each example: cd
  module2/examples/<name> && make
  clean && make run
- ./scripts/module2.sh runs the
  full module regression
- Compare output with MODULE2.md
  expected behavior notes

2. What to verify per feature

- ANSI ports — compile without separate input/output declarations
- always @* — output updates when any input toggles (no latch inference)
- generate — instance count matches parameter; signed adder handles negatives

Generate block — parameterized mux

```
/*
 * Parameterized mux with generate if - IEEE 1364-2001
 * generate for / generate if; elaboration-time logic.
 */

module mux_wide #(parameter WIDTH = 8) (
    input  wire [WIDTH-1:0] a,
    input  wire [WIDTH-1:0] b,
    input  wire             sel,
    output wire [WIDTH-1:0] y
);
    generate
        if (WIDTH == 1)
            assign y = sel ? b : a;
        else
            assign y = sel ? b : a;
# ...
```

Syllabus topics

Protocol & design details

Detail: What 1364-2001 Adds Over 1995 (1)

- ANSI-style port declarations: Direction and type (and size) in the port list
- **`always @*`**: Implicit sensitivity for combinational logic (no manual list)
- generate: Conditional and loop-based instantiation and logic
- signed: Signed arithmetic and signed nets/variables

Detail: What 1364-2001 Adds Over 1995 (2)

- Multi-dimensional arrays: e.g. `reg [7:0] mem [0:255]`
- localparam: Named constants that cannot be overridden
- ANSI-style task/function: Input/output in the header
- Other: File I/O improvements, `$signed`/`$unsigned`, attribute syntax, etc.

Detail: What 1364-2001 Still Does Not Include

- No SystemVerilog: no `logic`, `always_comb`/`always_ff`, interfaces, packages, classes
- No type parameters or interfaces (those are 1800)

Detail: ANSI-C Style Ports

- input wire: Default for input; can omit `wire` (e.g. `input [7:0] a`).
- output reg: Required when the output is driven in `always`/`initial`.
- output wire: When the output is driven only by `assign` or submodule.

Detail: Comparison: 1995 vs 2001 Port Style

- Same behavior; 2001 is more concise and keeps port and type in one place.

Detail: always @* Syntax

- **`@*` (or `@(*)`**): Sensitive to every signal read in the block (RHS and conditions).
- Use for combinational logic only; do not use for sequential (use `always @(posedge clk)`).

Detail: When to Use @* vs Explicit List

- **Prefer `@*`**: For combinational blocks; fewer errors, easier to maintain.
- Explicit list: When targeting strict 1995 compatibility or when a tool does not support `@*`.

Detail: generate / endgenerate

- genvar: Loop variable for generate loops; not a runtime variable.
- generate for: Replicates logic/instances; index must be constant at elaboration.

Detail: Conditional Generate

- generate if / else: Choose one of several implementations at elaboration.

Detail: signed Wire and reg

- signed: Affects arithmetic and comparison in expressions involving that net/reg.
- Unsigned (default): No sign extension; values 0 to 2^N-1.

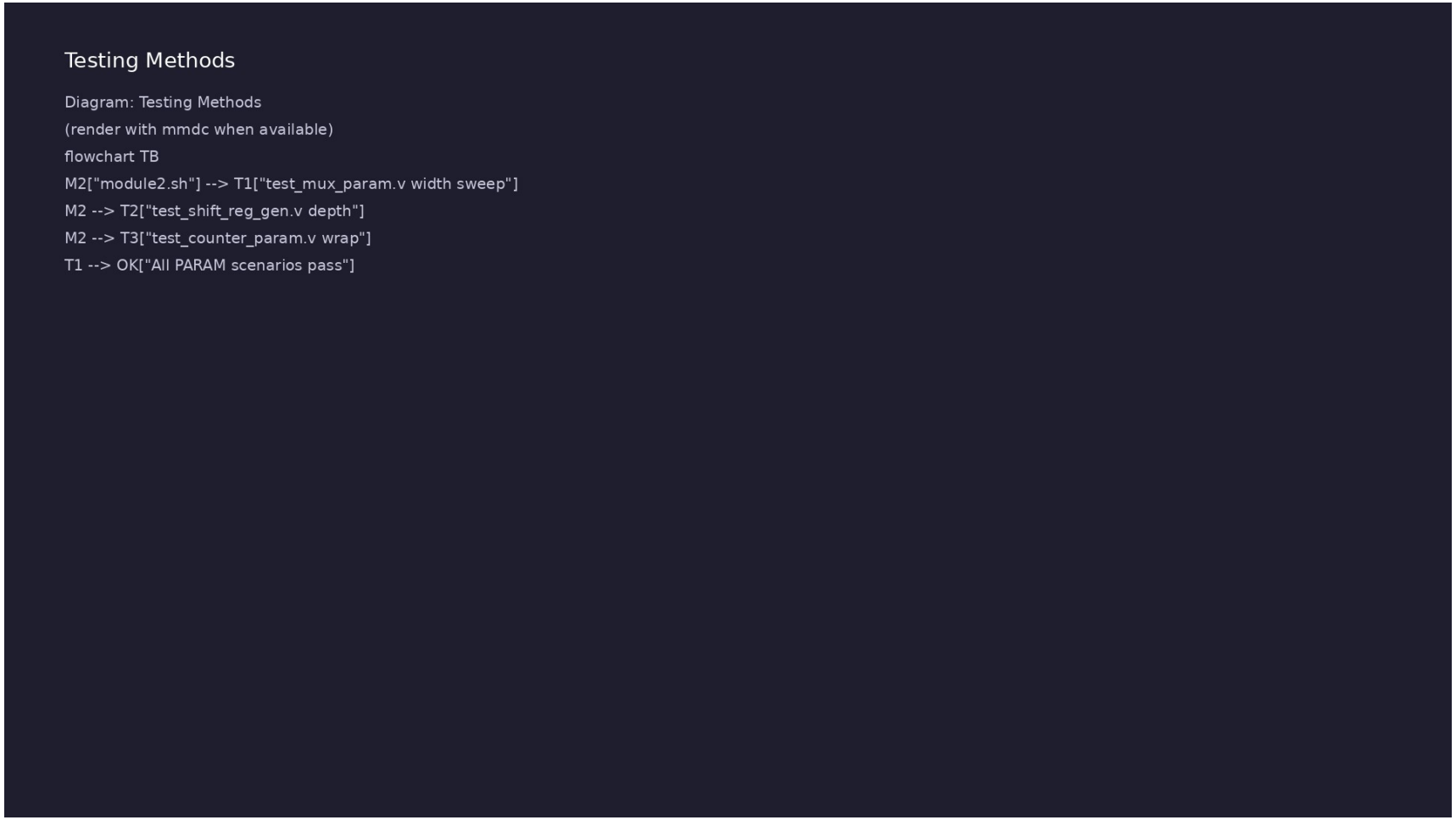
Detail: Declaration and Usage

- Packed (left): `[7:0]` — contiguous bits, can be used as a single vector.
- Unpacked (right): `[0:255]` — array of elements; select with one index at a time in 2001.

Detail: Task (2001 ANSI)

- automatic: Optional; gives automatic storage (re-entrant). Useful for recursive or concurrent calls.
- input/output in header: Cleaner than 1995 style; avoids duplicate declarations.
- Note: In 1364-2001 use `output reg` (not `output logic`; `logic` is SystemVerilog).

Verification flow (reference)



Stimulus, DUT response, and check points for this module.

1. IEEE 1364-2001 Context (1/2)

- ANSI-style port declarations: Direction and type (and size) in the port list
- **`always @*`**: Implicit sensitivity for combinational logic (no manual list)
- generate: Conditional and loop-based instantiation and logic
- signed: Signed arithmetic and signed nets/variables
- Multi-dimensional arrays: e.g. `reg [7:0] mem [0:255]`
- localparam: Named constants that cannot be overridden

1. IEEE 1364-2001 Context (2/2)

- ANSI-style task/function: Input/output in the header
- Other: File I/O improvements, `$signed`/`$unsigned`, attribute syntax, etc.
- No SystemVerilog: no `logic`, `always_comb`/`always_ff`, interfaces, packages, classes
- No type parameters or interfaces (those are 1800)

2. ANSI-Style Port Declarations (2001)

- input wire: Default for input; can omit `wire` (e.g. `input [7:0] a`).
- output reg: Required when the output is driven in `always`/`initial`.
- output wire: When the output is driven only by `assign` or submodule.
- Same behavior; 2001 is more concise and keeps port and type in one place.

3. Implicit Sensitivity: always @* (2001)

- **`@*` (or `@(*)`**): Sensitive to every signal read in the block (RHS and conditions).
- Use for combinational logic only; do not use for sequential (use `always @(posedge clk)`).
- **Prefer `@*`**: For combinational blocks; fewer errors, easier to maintain.
- Explicit list: When targeting strict 1995 compatibility or when a tool does not support `@*`.

4. Generate (2001)

- genvar: Loop variable for generate loops; not a runtime variable.
- generate for: Replicates logic/instances; index must be constant at elaboration.
- generate if / else: Choose one of several implementations at elaboration.
- generate / endgenerate
- Conditional Generate

5. Signed Types (2001)

- signed: Affects arithmetic and comparison in expressions involving that net/reg.
- Unsigned (default): No sign extension; values 0 to 2^N-1.
- signed Wire and reg
- $signed and $unsigned

6. Multi-Dimensional Arrays (2001)

- Packed (left): `[7:0]` — contiguous bits, can be used as a single vector.
- Unpacked (right): `[0:255]` — array of elements; select with one index at a time in 2001.
- Declaration and Usage

7. localparam (2001)

- parameter: Can be overridden: `#(.WIDTH(16))`.
- localparam: Cannot be overridden; often derived from parameters or fixed.

8. ANSI-Style Tasks and Functions (2001)

- automatic: Optional; gives automatic storage (re-entrant). Useful for recursive or concurrent calls.
- input/output in header: Cleaner than 1995 style; avoids duplicate declarations.
- Note: In 1364-2001 use `output reg` (not `output logic`; `logic` is SystemVerilog).
- Function (2001 ANSI)
- Task (2001 ANSI)

Hands-on examples

Module 2 toolchain check

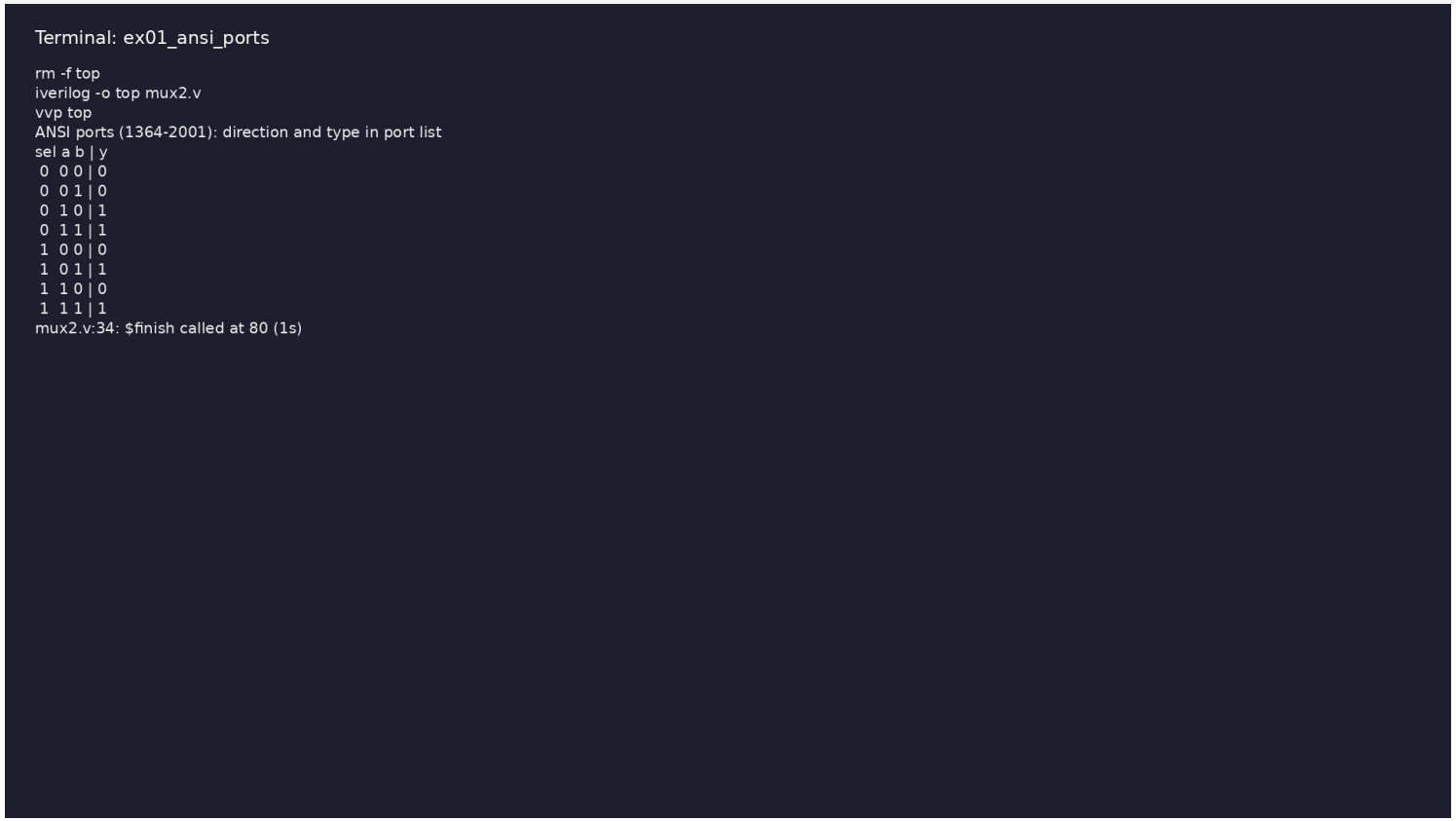
```
$ command -v iverilog >/dev/null && echo "Toolchain ready: $(iverilog
      -V 2>&1 | head -1)"
```

Example 1: Module and instantiation with ANSI-style ports

- Port direction and type in port list

Demo: Module and instantiation with ANSI-style ports

```
$ cd module2/examples/ansi_ports && make clean && make run
```



Example 2: Combinational blocks with implicit sensitivity

- No manual sensitivity list; tool infers from RHS

Demo: Combinational blocks with implicit sensitivity

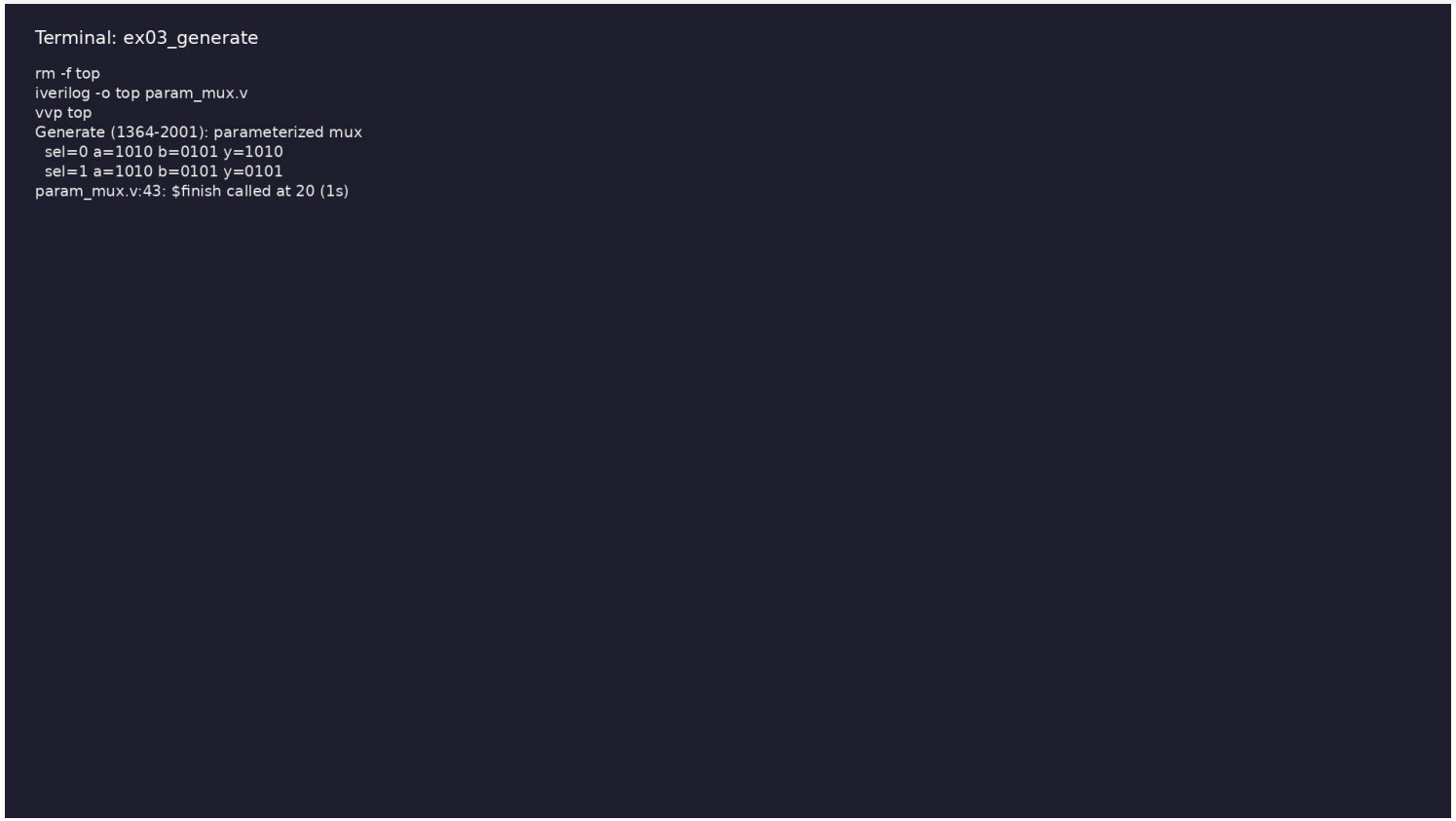
```
$ cd module2/examples/procedural && make clean && make run
```

Example 3: Parameterized mux; generate-style parameterization

- genvar, elaboration-time logic

Demo: Parameterized mux; generate-style parameterization

```
$ cd module2/examples/generate && make clean && make run
```

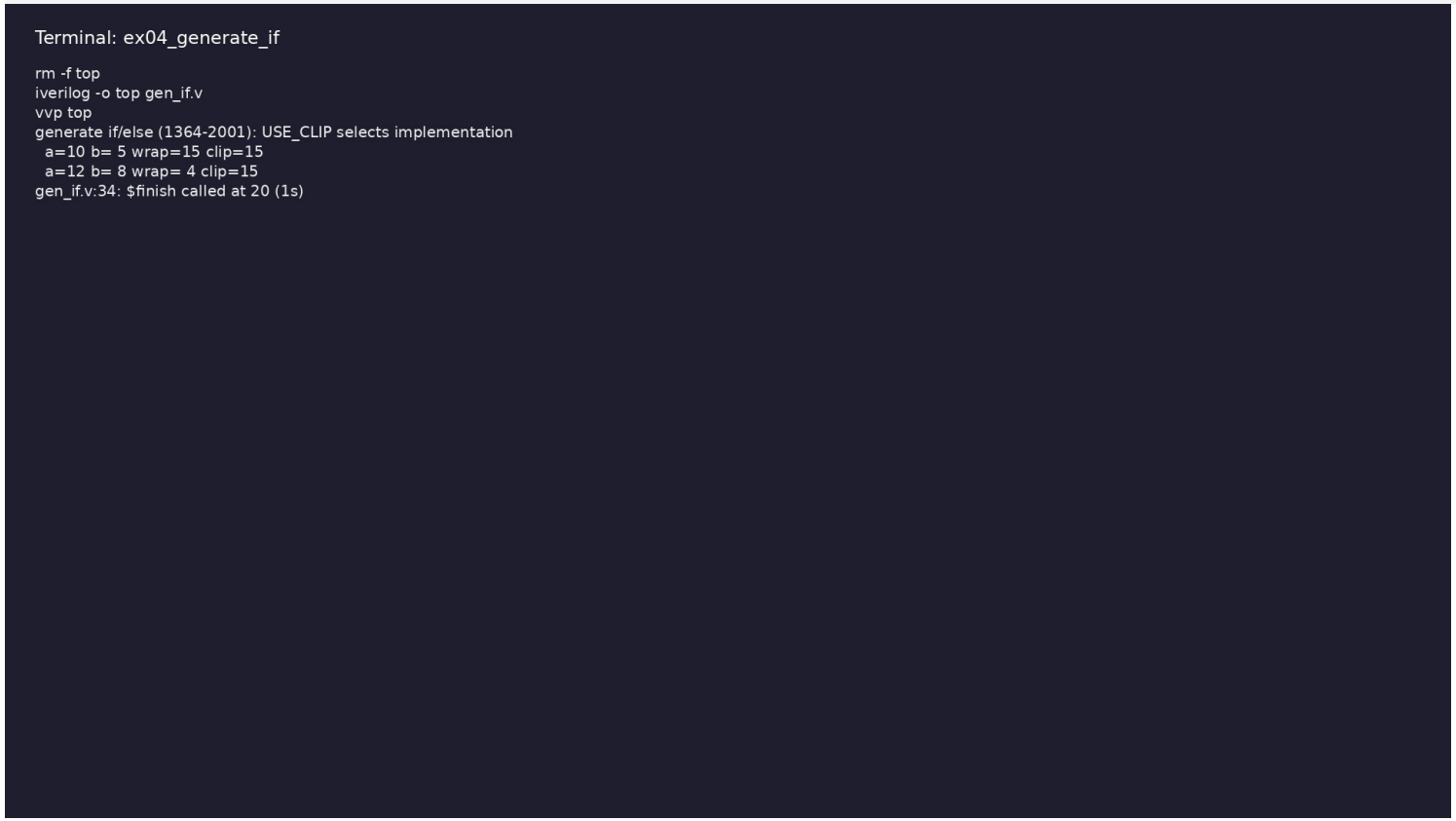


Example 4: Conditional implementation by parameter (e.g. USE_CLIP wra…

- generate if/else at elaboration

Demo: Conditional implementation by parameter (e.g. USE_CLIP wrap vs…

```
$ cd module2/examples/generate_if && make clean && make run
```



Example 5: N-bit ripple-carry adder using generate for (full_adder in…

- genvar loop, replicated hierarchy

Demo: N-bit ripple-carry adder using generate for (full_adder instanc…

```
$ cd module2/examples/generate_ripple_adder && make clean && make run
```

Example 6: signed reg/wire, $signed/$unsigned

- Two's complement arithmetic in RTL

Demo: signed reg/wire, $signed/$unsigned

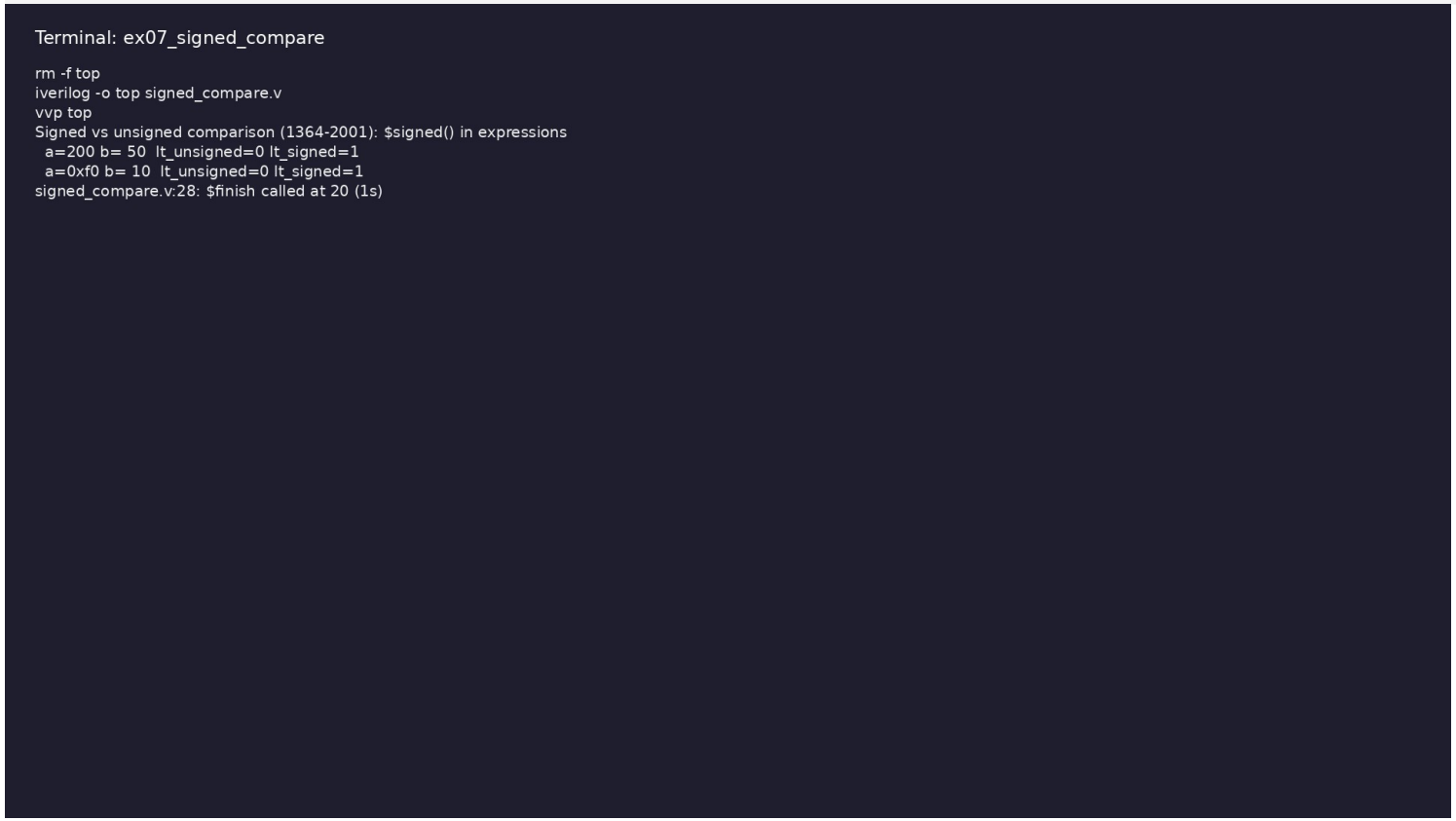
```
$ cd module2/examples/signed && make clean && make run
```

Example 7: $signed() in expressions; a < b unsigned vs signed

- Cast in expressions without changing declaration

Demo: $signed() in expressions; a < b unsigned vs signed

```
$ cd module2/examples/signed_compare && make clean && make run
```



Example 8: 2:4 decoder with ANSI ports and always @* (case)

- Combinational case; one-hot output

Demo: 2:4 decoder with ANSI ports and always @* (case)

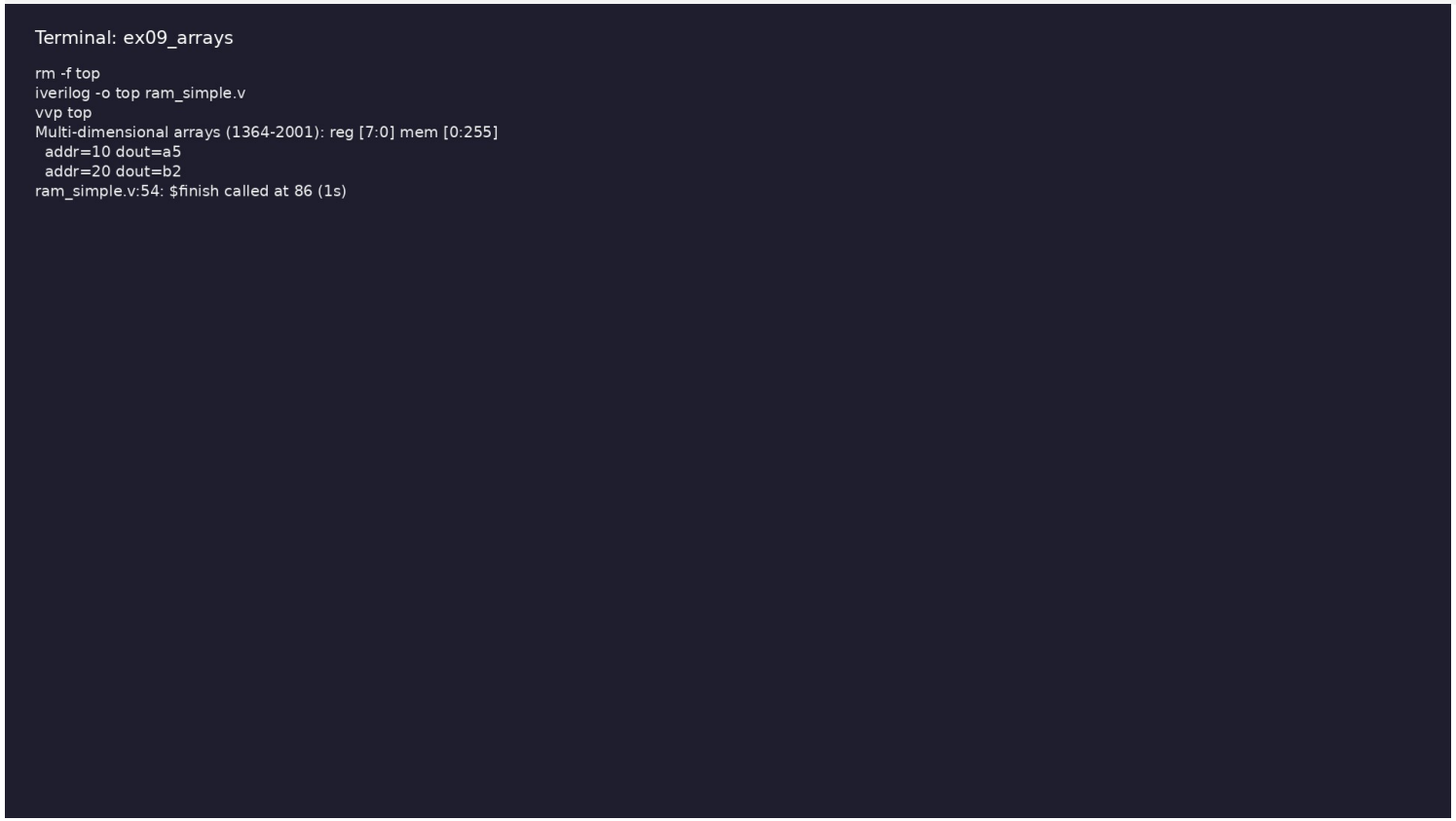
```
$ cd module2/examples/decoder && make clean && make run
```

Example 9: Memory-style arrays (reg [7:0] mem [0:255]), indexing

- Packed vs unpacked; array indexing rules in 2001

Demo: Memory-style arrays (reg [7:0] mem [0:255]), indexing

```
$ cd module2/examples/arrays && make clean && make run
```

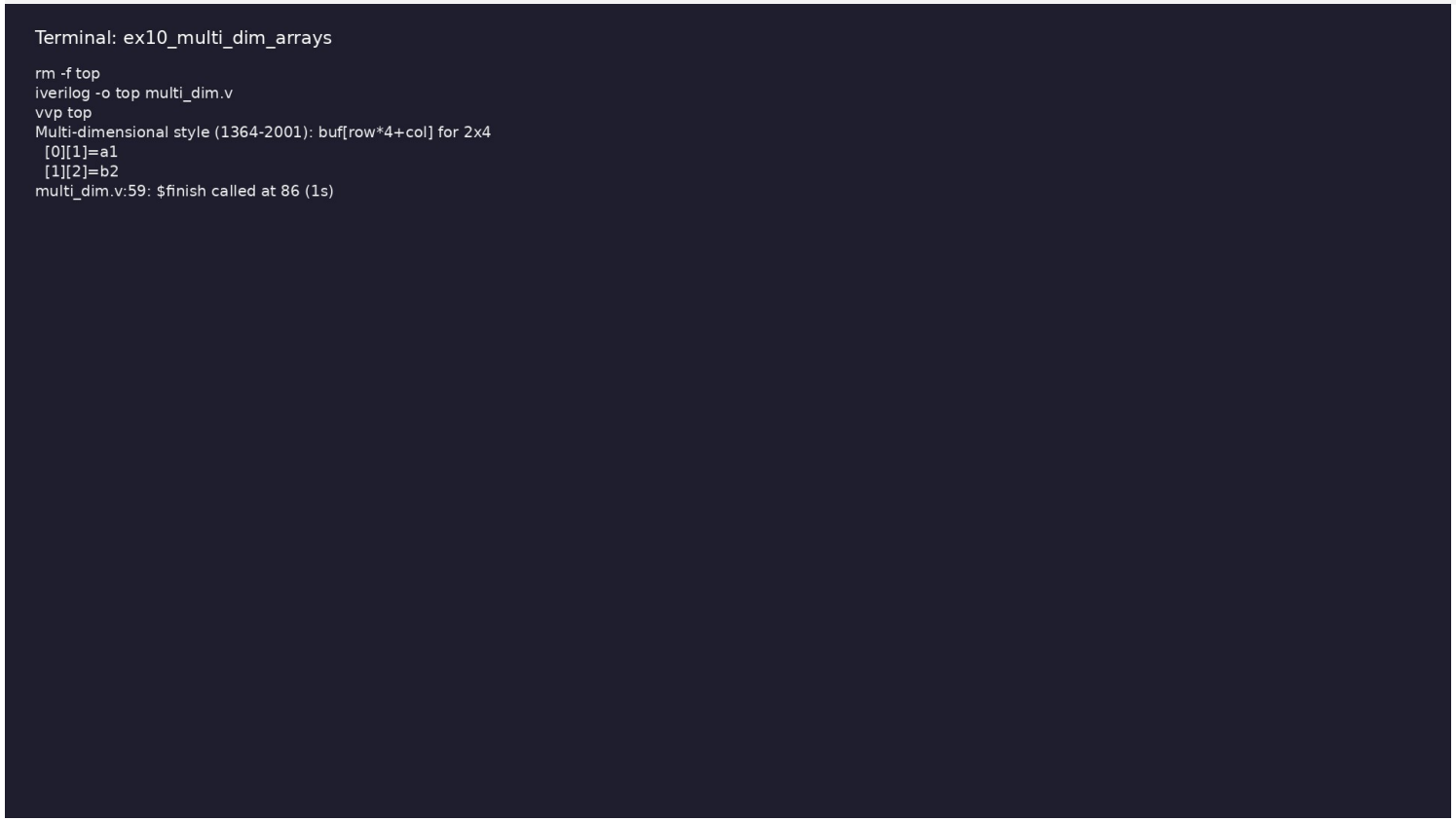


Example 10: 2x4 buffer using 1D array indexed as row*4+col

- Emulating 2D with 1D array; index expression

Demo: 2x4 buffer using 1D array indexed as row*4+col

```
$ cd module2/examples/multi_dim_arrays && make clean && make run
```



Example 11: Parameter override, localparam for derived constants

- When to use parameter vs localparam

Demo: Parameter override, localparam for derived constants

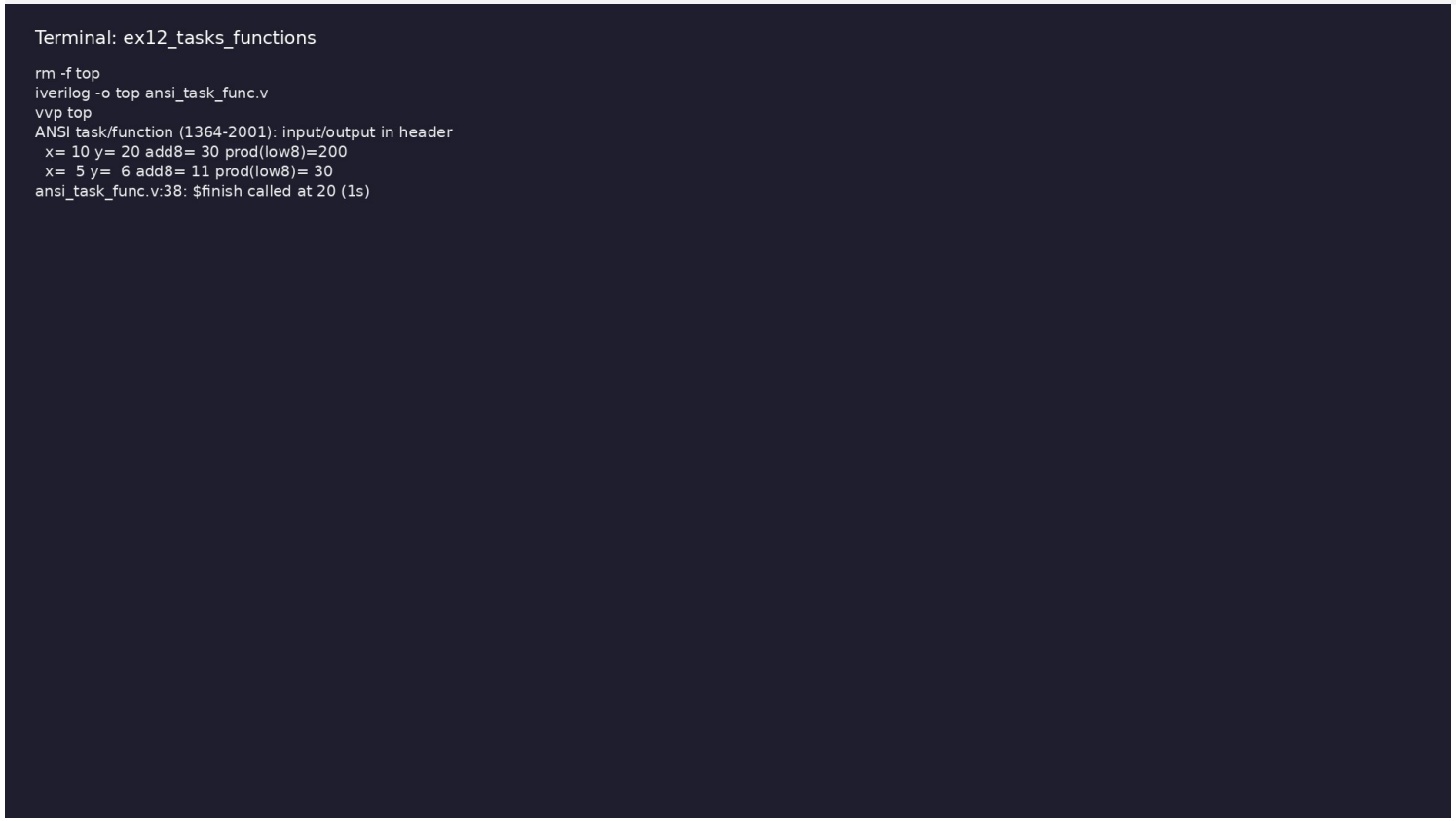
```
$ cd module2/examples/parameters && make clean && make run
```

Example 12: Input/output in header, automatic functions

- 2001 style vs 1995 style

Demo: Input/output in header, automatic functions

```
$ cd module2/examples/tasks_functions && make clean && make run
```



Example 13: task automatic apply_reset(output reg rn); begin/end body

- Task with output argument; testbench reset pattern

Demo: task automatic apply_reset(output reg rn); begin/end body

```
$ cd module2/examples/task_ansi && make clean && make run
```

Common pitfalls

Using always @* for Sequential Logic

- Mistake: `always @* q <= d;` intending a flip-flop.
- Reality: `@*` is for combinational logic; it does not create a clock edge.
- Correct: Use `always @(posedge clk)` (and `<=`) for sequential logic.
- Why: Mixing the two causes simulation/synthesis mismatches and unintended latches.

genvar in Normal Code

- Mistake: Using genvar in an always block or initial block.
- Reality: genvar is only for generate loops; it does not exist at runtime.
- Correct: Use integer or reg for runtime loops; use genvar only inside generate for.
- Why: genvar is elaborated away; it is not a simulation variable.

Overriding localparam

- Mistake: Trying to override localparam at instantiation (e.g. `#(.MAX(100))`).
- Reality: localparam cannot be overridden.
- Correct: Use parameter for values that must be overridden; use localparam for fixed or derived cons…
- Why: The standard does not allow localparam in the parameter list.

Signed vs Unsigned Mix

- Mistake: Mixing signed and unsigned in one expression without $signed/$unsigned and expecting a par…
- Reality: Rules for sign extension and comparison depend on operands.
- Correct: Be explicit: use signed types or $signed/$unsigned and verify with testbenches.
- Why: Implicit mixing leads to wrong arithmetic and comparison results.

Generate Block Scope

- Mistake: Referencing generate block names or genvar at runtime.
- Reality: Generate blocks are elaborated at compile time; names are for hierarchy.
- Correct: Use generate for structure; use normal variables and blocks for runtime behavior.
- Why: Generate is not “runtime”; it only affects what gets instantiated.

Practice & assessment

What you should know (1/2)

- ✓ Write modules with ANSI-style port declarations (1364-2001)
- ✓ Use `always @*` for combinational logic and avoid sensitivity-list errors
- ✓ Use generate (for, if/else) for parameterized and conditional RTL
- ✓ Use signed types and $signed/$unsigned for signed arithmetic
- ✓ Declare and use multi-dimensional arrays (e.g. memories)
- ✓ Use localparam for constants that must not be overridden

What you should know (2/2)

- ✓ Write ANSI-style tasks and functions
- ✓ Compare 1364-1995 and 1364-2001 and list what 2001 adds

Exercises

- ANSI Ports
- always @
- Generate
- Signed
- Arrays and localparam

Exercises

- Using always @* for Sequential Logic
- genvar in Normal Code
- Overriding localparam
- Signed vs Unsigned Mix
- Generate Block Scope

Summary & next steps

- Master the major Verilog update (IEEE 1364-2001) and its additions over 1364-1995: ANSI ports, impl…
- Complete module2/CHECKLIST.md
- Review module2/EXAMPLES.md and run each lab
- Next: Next module in course